

Lab 9 – Bottom Parser for Simple Grammar

Q1.c

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#define STATES 12
#define SYMBOLS 9
#define MAX_INPUT 100
char table[STATES][SYMBOLS][10] = {
{"s5", "", "", "s4", "", "", "1", "2", "3"},
{"", "s6", "", "", "", "acc", "", "", ""},
{"", "r2", "s7", "", "r2", "r2", "", "", ""},
{"", "r4", "r4", "", "r4", "r4", "", "", ""},
{"s5", "", "", "s4", "", "", "8", "2", "3"},
{"", "r6", "r6", "", "r6", "r6", "", "", ""},
{"s5", "", "", "s4", "", "", "", "9", "3"},
{"s5", "", "", "s4", "", "", "", "", "10"},
{"", "s6", "", "", "r1", "", "", "", ""},
{"", "r1", "s7", "", "r1", "r1", "", "", ""},
{"", "r3", "r3", "", "r3", "r3", "", "", ""},
{"", "r5", "r5", "", "r5", "r5", "", "", ""}
};
char *lhs[] = {"", "E", "E", "T", "T", "F", "F"};
int rhs_len[] = {0, 3, 1, 3, 1, 3, 1};
int get_col(char *sym) {
if (strcmp(sym, "id") == 0) return 0;
if (strcmp(sym, "+") == 0) return 1;
if (strcmp(sym, "*") == 0) return 2;
if (strcmp(sym, "(") == 0) return 3;
if (strcmp(sym, ")") == 0) return 4;
if (strcmp(sym, "$") == 0) return 5;
if (strcmp(sym, "E") == 0) return 6;
if (strcmp(sym, "T") == 0) return 7;
if (strcmp(sym, "F") == 0) return 8;
return -1;
}
void parse(char input[][10], int n) {
int stack[100];
int top = 0;
stack[top] = 0;
int i = 0;
printf("\n%-20s %-20s %-10s\n", "Stack", "Input", "Action");
while (1) {
```

```

int state = stack[top];
int col = get_col(input[i]);
if (col == -1) {
    printf("Invalid symbol: %s\n", input[i]);
    return;
}
char action[10];
strcpy(action, table[state][col]);
for (int j = 0; j <= top; j++)
    printf("%d ", stack[j]);
printf("\t\t");
for (int j = i; j < n; j++)
    printf("%s ", input[j]);
printf("\t\t%s\n", action);
if (strlen(action) == 0) {
    printf("\nParsing Error\n");
    return;
}
if (strcmp(action, "acc") == 0) {
    printf("\nString Accepted\n");
    return;
}
if (action[0] == 's') {
    int next = atoi(action + 1);
    stack[++top] = next;
    i++;
}
else if (action[0] == 'r') {
    int prod = atoi(action + 1);
    top -= rhs_len[prod];
    int new_state = stack[top];
    int goto_col = get_col(lhs[prod]);
    int next = atoi(table[new_state][goto_col]);
    stack[++top] = next;
}
}
}
}

int main() {
    char line[200];
    char input[MAX_INPUT][10];
    int n = 0;
    printf("Enter input tokens separated by space (end with $):\n");
    fgets(line, sizeof(line), stdin);
    char *token = strtok(line, " \n");
    while (token != NULL) {

```

```

strcpy(input[n++], token);
token = strtok(NULL, " \n");
}
parse(input, n);
return 0;
}

```

Output:

```
6CSEC1@db1-23:~/Documents/CD_230905152/L9$ cc -o abc q1.c
```

```
6CSEC1@db1-23:~/Documents/CD_230905152/L9$ ./abc
```

Enter input tokens separated by space (end with \$):

```
id * $
```

Stack	Input	Action
0	id * \$	s5
0 5	* \$	r6
0 3	* \$	r4
0 2	* \$	s7
0 2 7	\$	

Parsing Error

```
6CSEC1@db1-23:~/Documents/CD_230905152/L9$ ./abc
```

Enter input tokens separated by space (end with \$):

```
id + id * id $
```

Stack	Input	Action
0	id + id * id \$	s5
0 5	+ id * id \$	r6
0 3	+ id * id \$	r4
0 2	+ id * id \$	r2
0 1	+ id * id \$	s6
0 1 6	id * id \$	s5
0 1 6 5	* id \$	r6
0 1 6 3	* id \$	r4
0 1 6 9	* id \$	s7
0 1 6 9 7	id \$	s5
0 1 6 9 7 5	\$	r6
0 1 6 9 7 10	\$	r3
0 1 6 9	\$	r1
0 1	\$	acc

String Accepted